Data Mining and Neural Networks

# Tree Ensembles

Week 03

**Muhammad Ateeq**

MA3022/MA4022/MA7022

School of Computing and Mathematical Sciences
University of Leicester

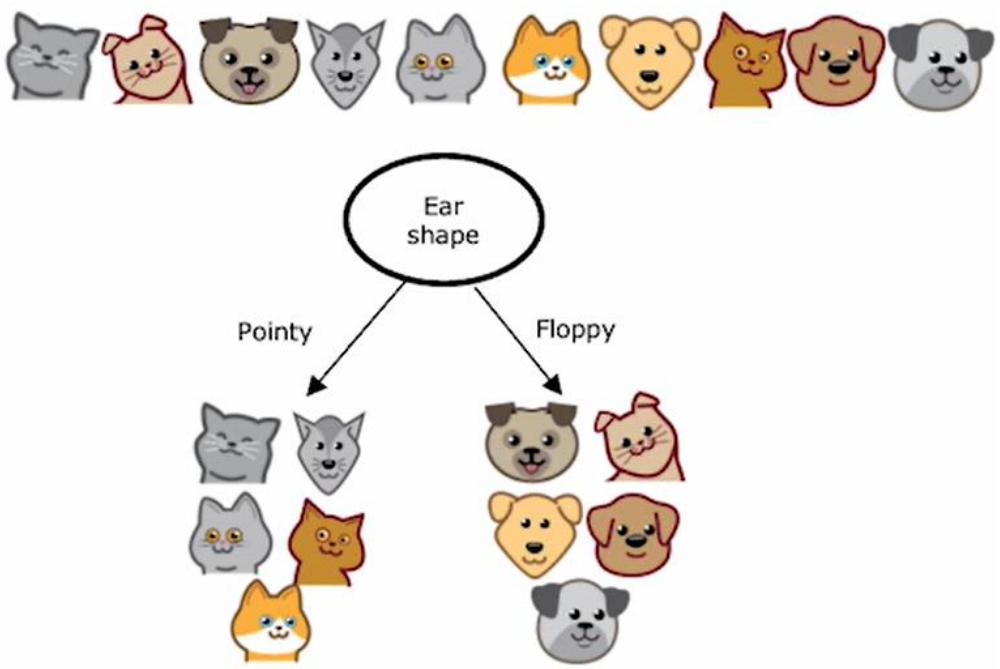
27/28/30-Jan-2026

Credits: Andrew Ng

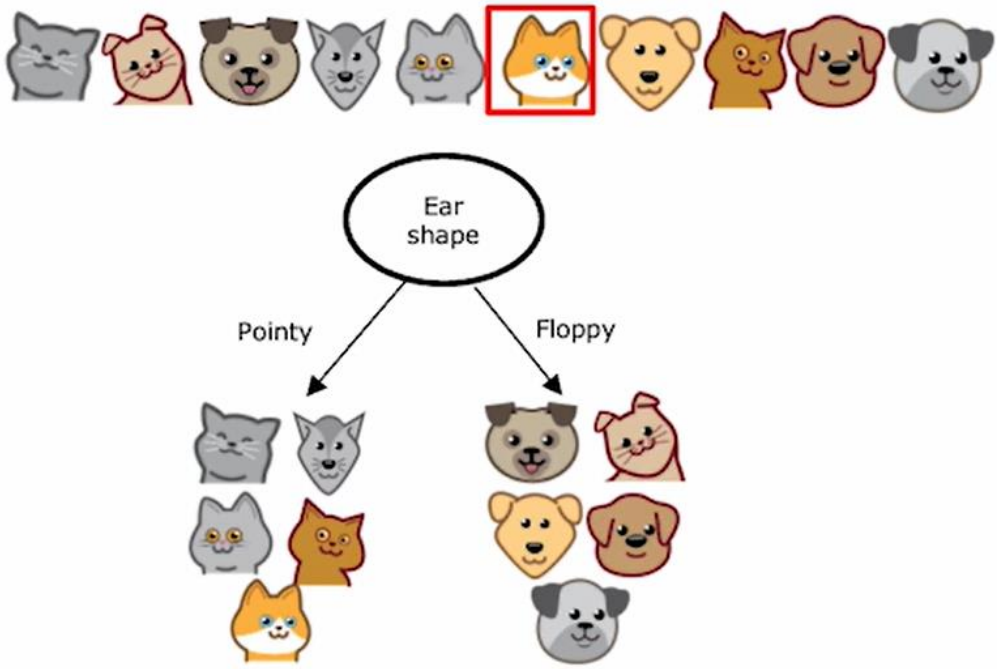
# Agenda

- Decision trees
  - **Tree ensembles**

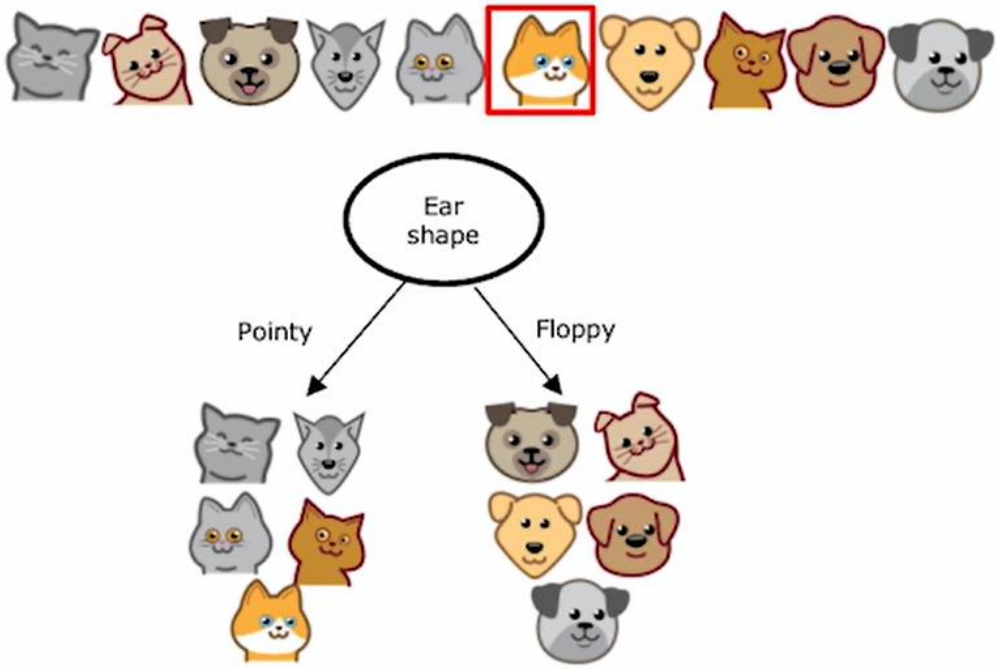
# Trees are highly sensitive to small changes of the data

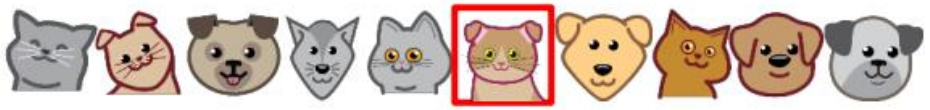


# Trees are highly sensitive to small changes of the data

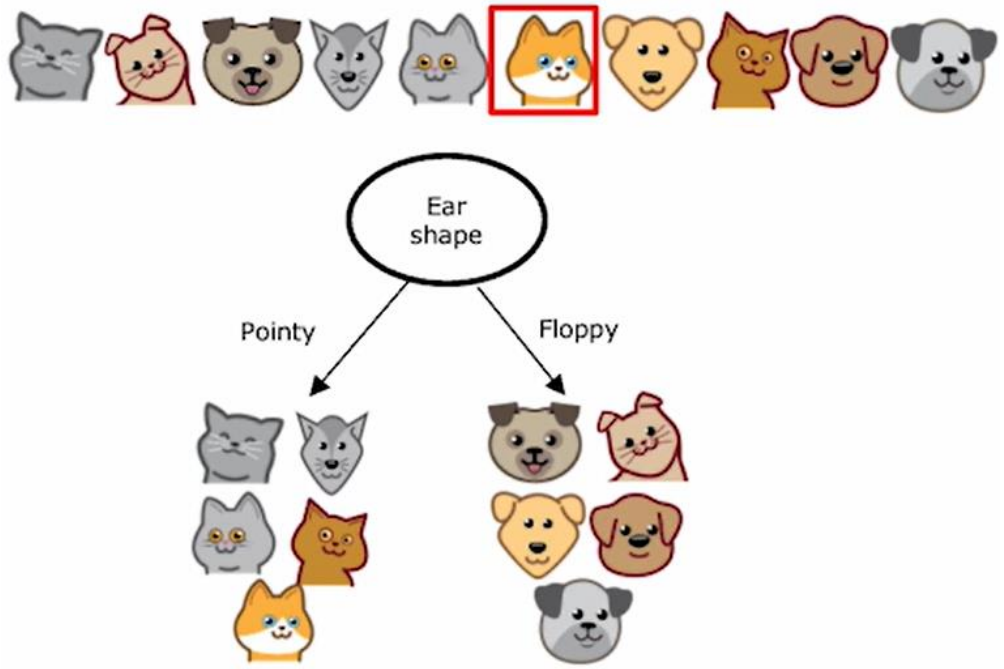


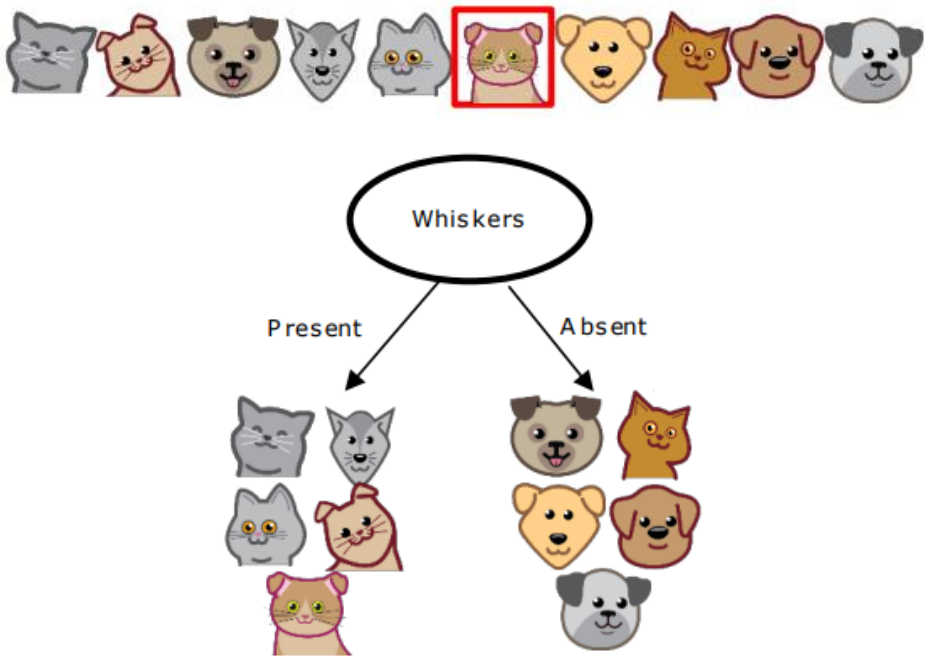
# Trees are highly sensitive to small changes of the data



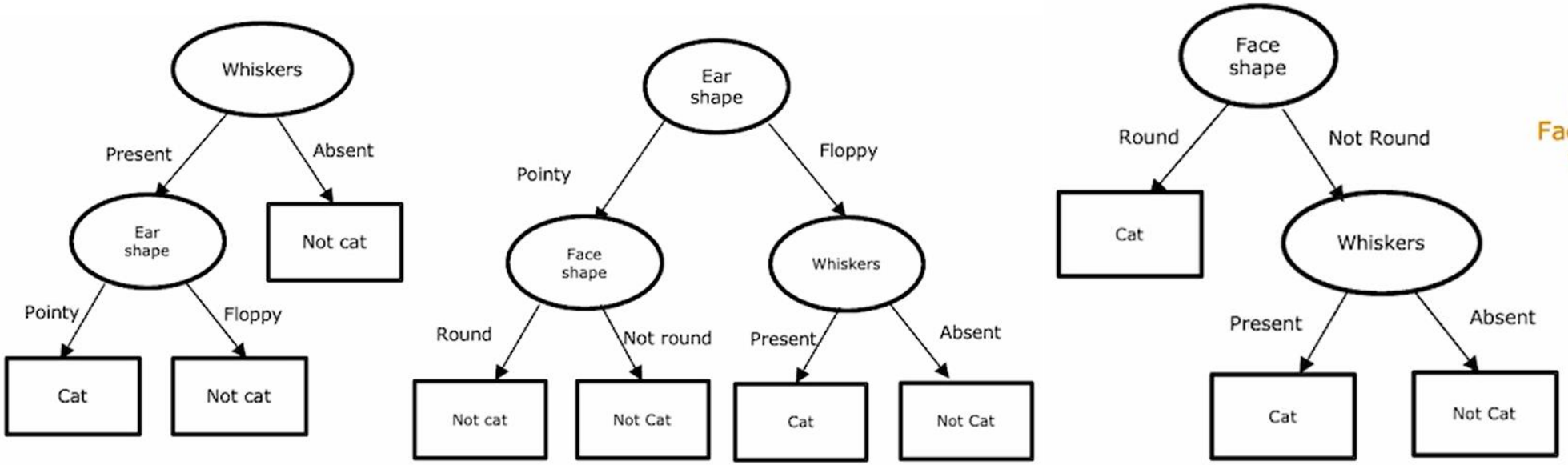



# Trees are highly sensitive to small changes of the data



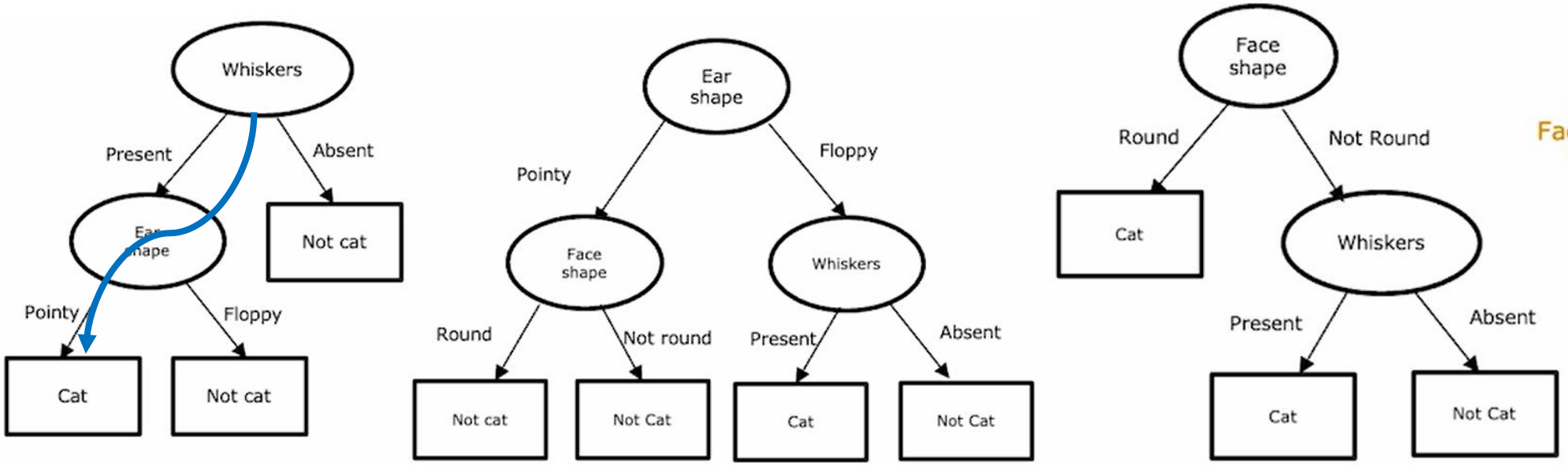
# Tree Ensemble




New test example

Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

# Tree Ensemble

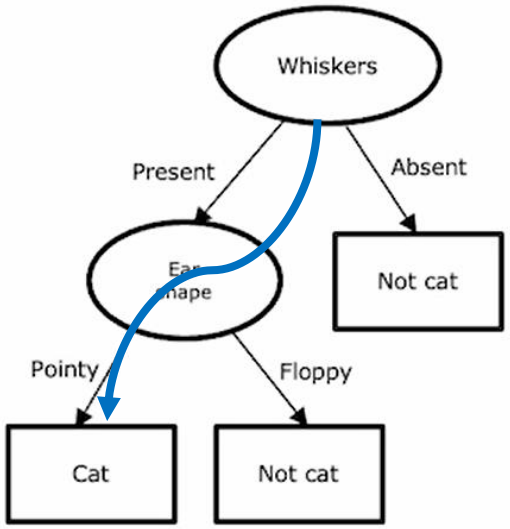


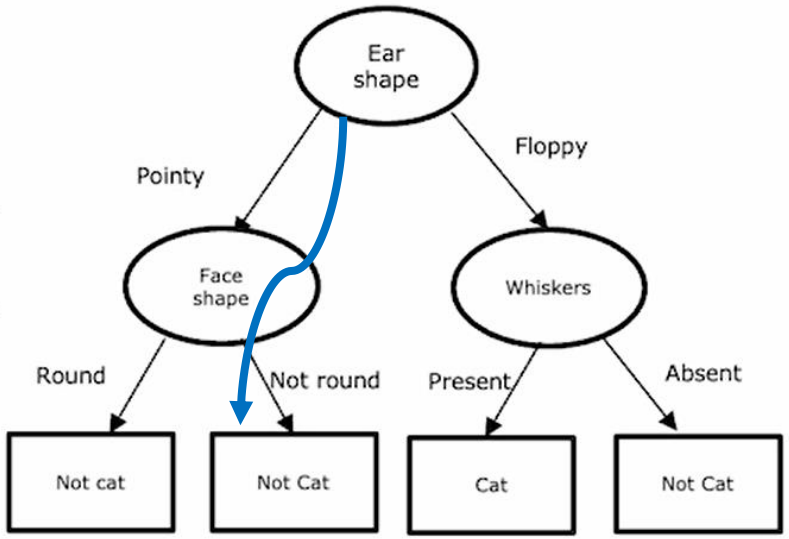

New test example

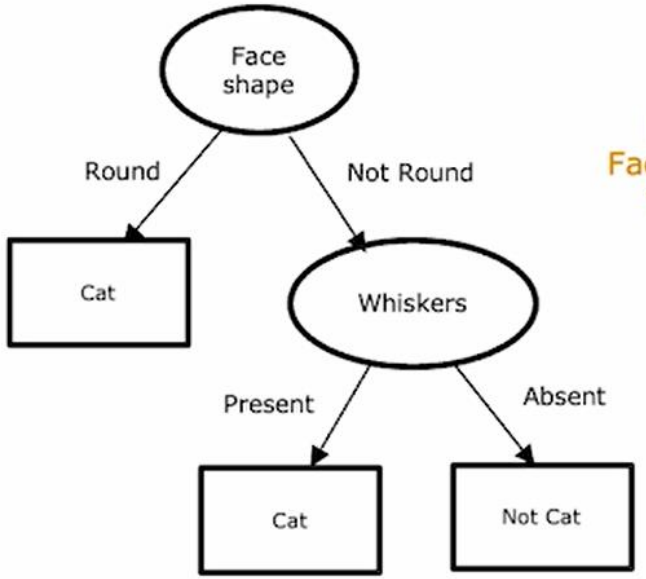
Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

Prediction: Cat

# Tree Ensemble



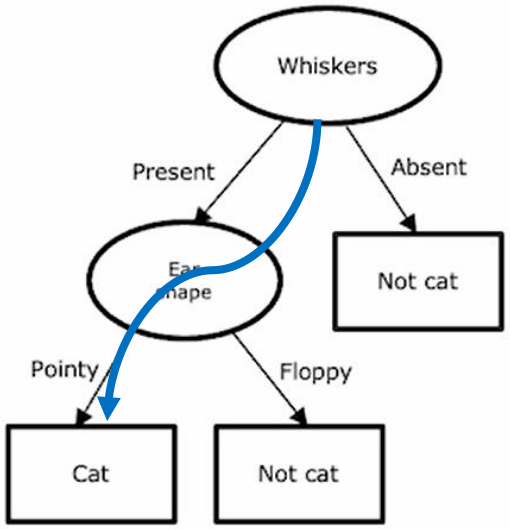

New test example

Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

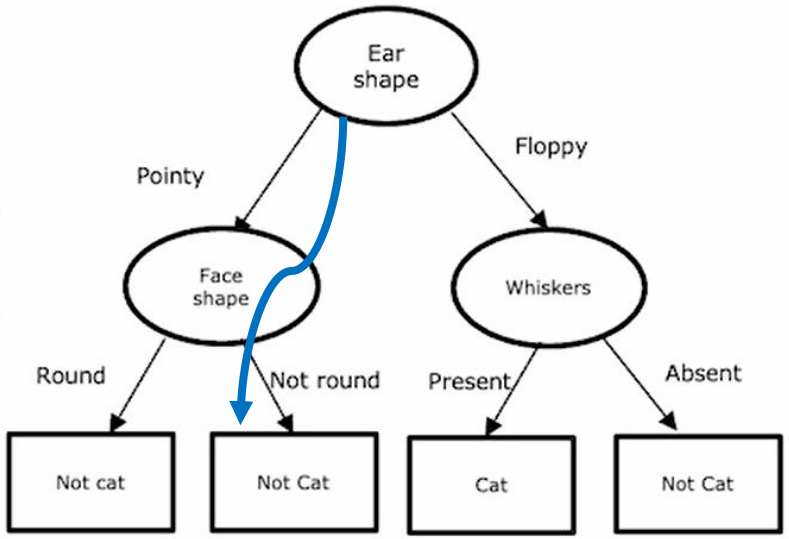
Prediction: Cat

Prediction: Not cat

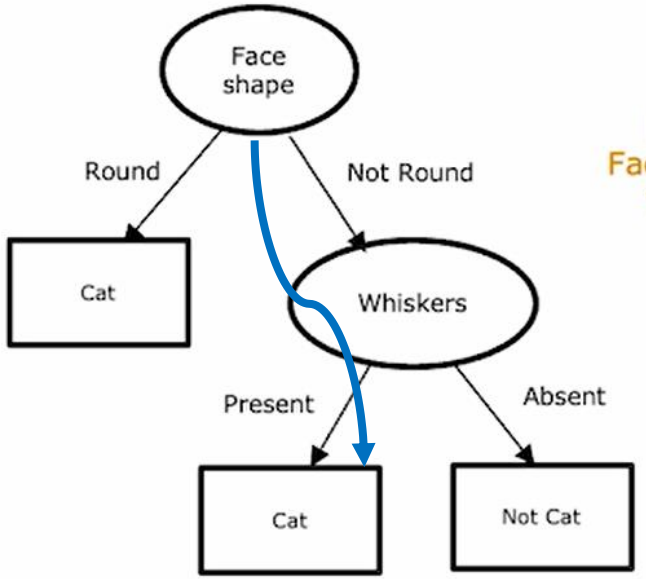
# Tree Ensemble




Prediction: Cat




Prediction: Not cat



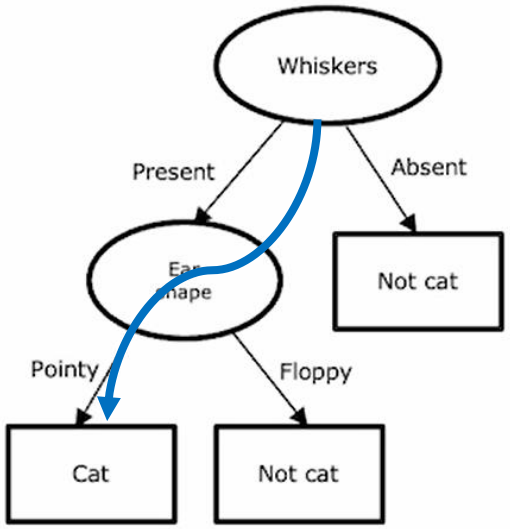

Prediction: Cat

New test example

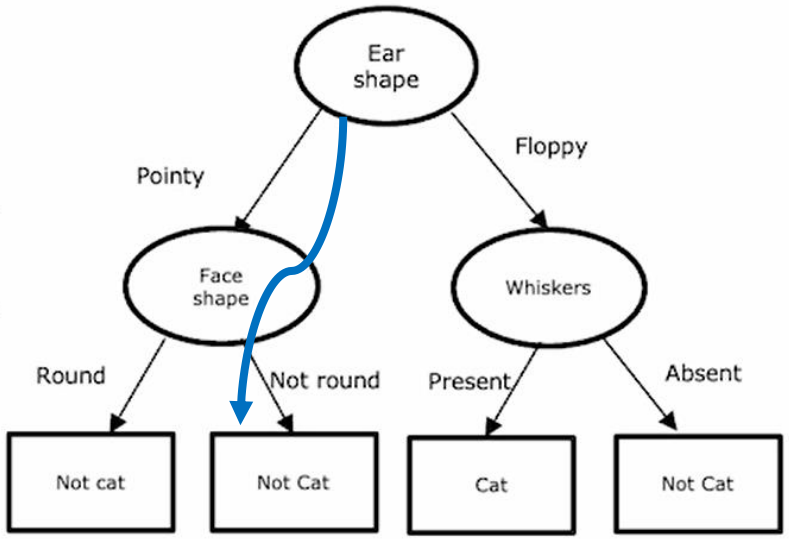
Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

UNIVERSITY OF LEICESTER

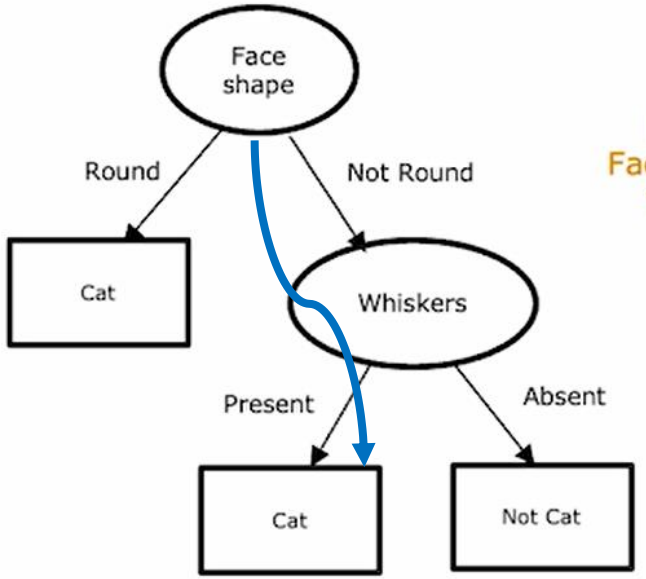
# Tree Ensemble




Prediction: Cat




Prediction: Not cat




Prediction: Cat

New test example

Ear shape: Pointy
Face shape: Not Round
Whiskers: Present

Final prediction: Cat

UNIVERSITY OF LEICESTER

# Agenda

- Decision trees
  - Tree ensembles
  - **Sampling with replacement**

# Sampling with Replacement

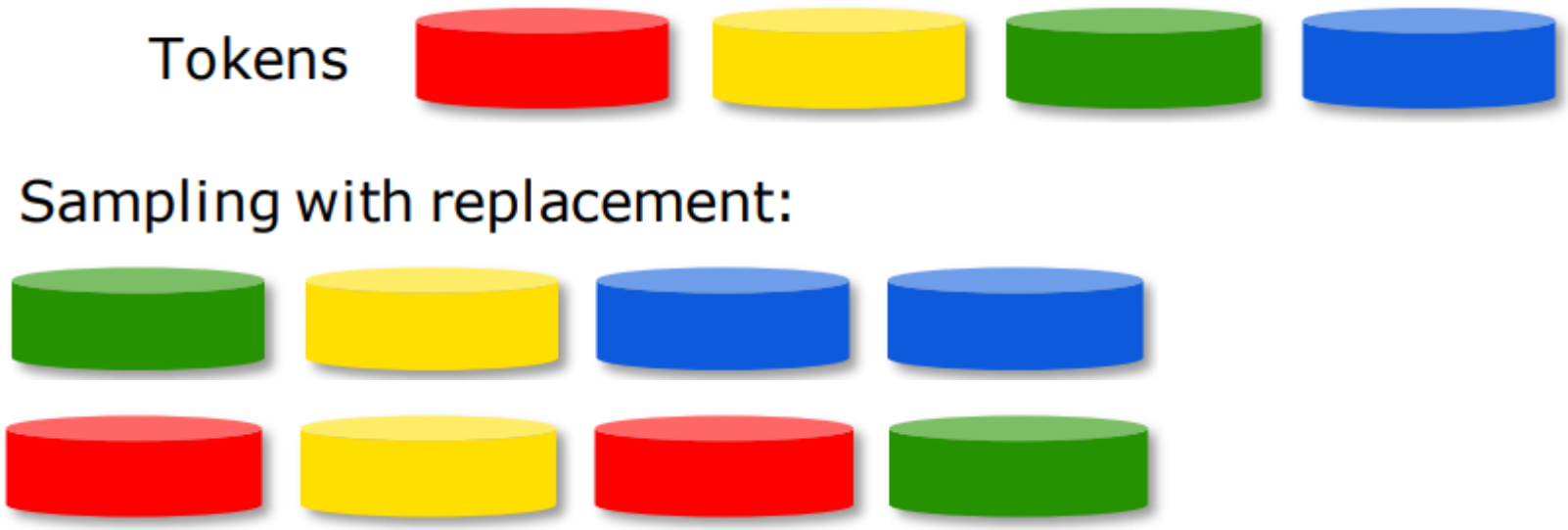
Tokens

# Sampling with Replacement

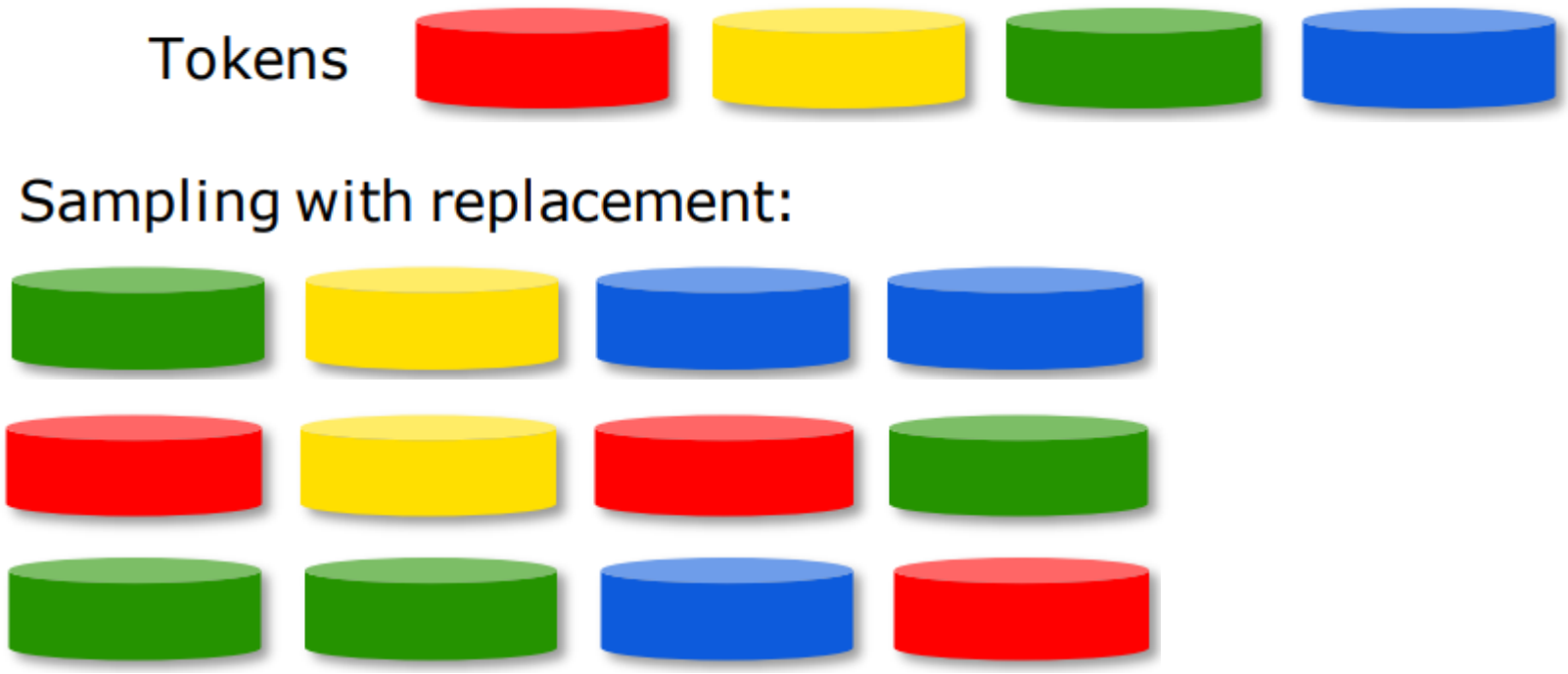
Tokens

Sampling with replacement:

# Sampling with Replacement



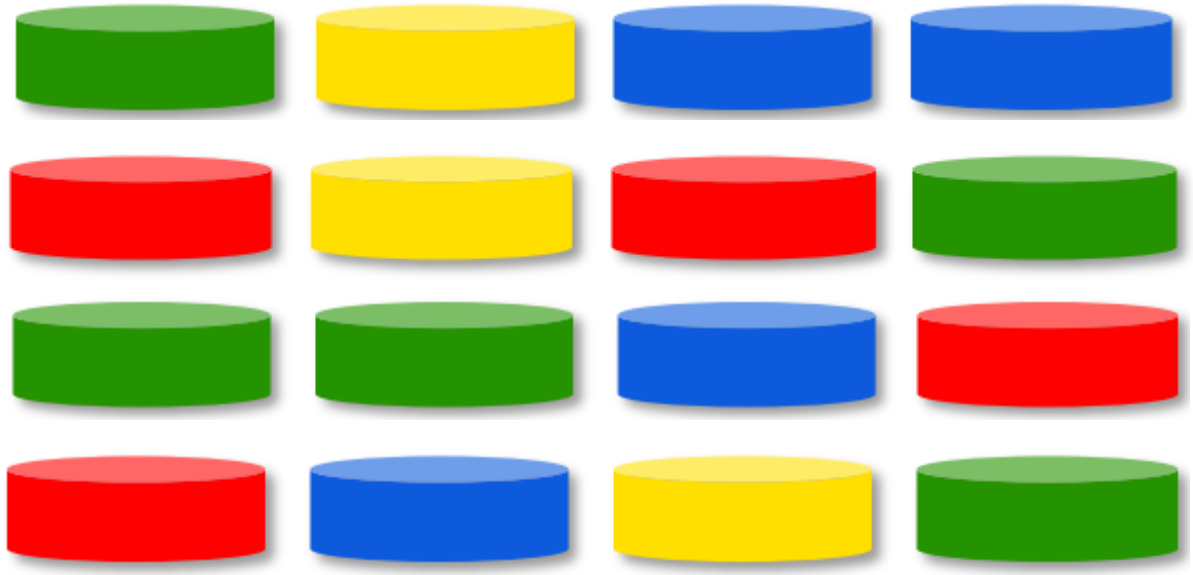
# Sampling with Replacement



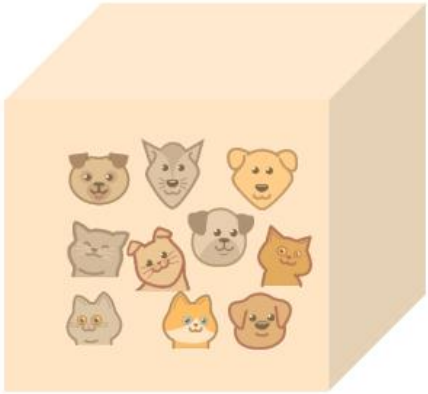
# Sampling with Replacement

Tokens

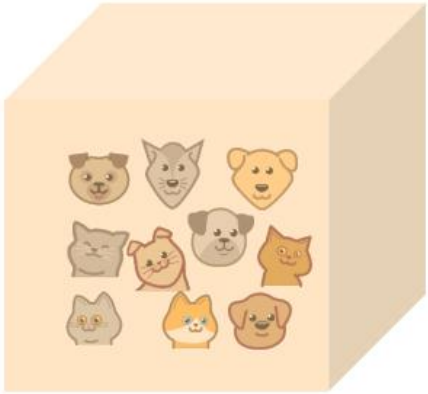
Sampling with replacement:



# Sampling with Replacement

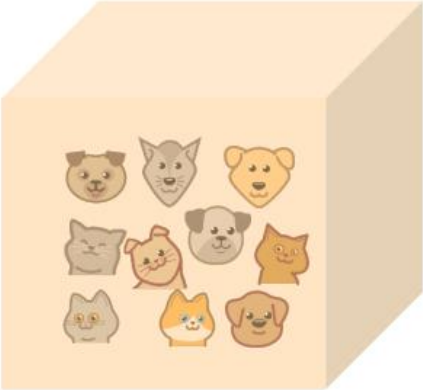


# Sampling with Replacement



| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

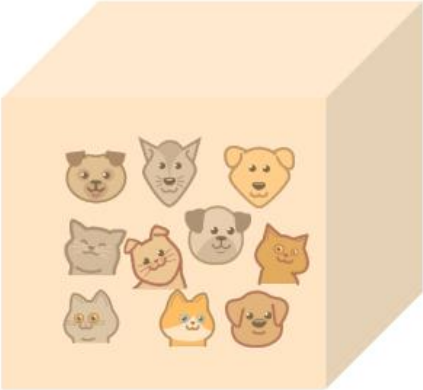
# Sampling with Replacement



| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

UNIVERSITY OF LEICESTER

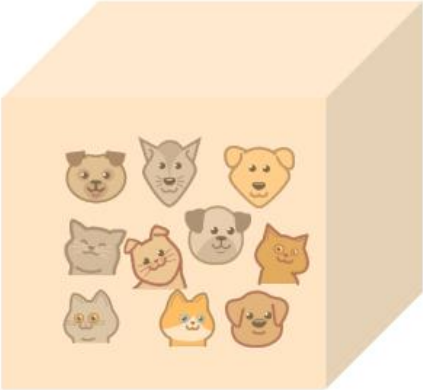
# Sampling with Replacement



| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | 1 |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

UNIVERSITY OF LEICESTER

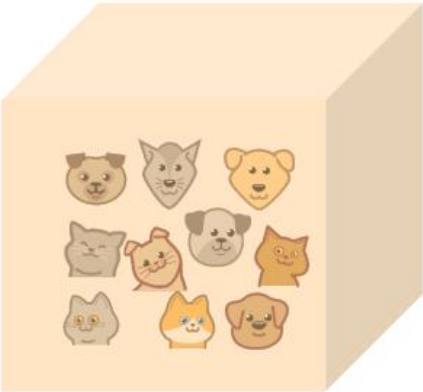
# Sampling with Replacement



| | Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|---|
| | Pointy | Round | Present | 1 |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

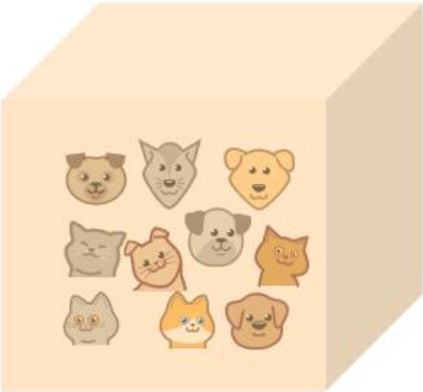
UNIVERSITY OF LEICESTER

# Sampling with Replacement



| | Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|---|
| | Pointy | Round | Present | 1 |
| | Floppy | Not round | Absent | 0 |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

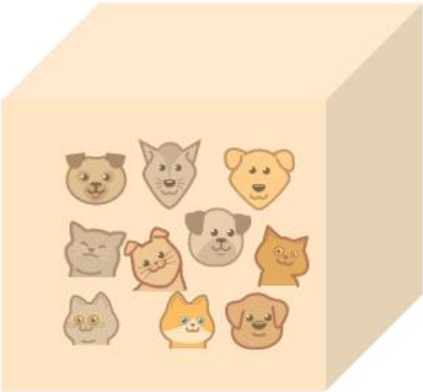
# Sampling with Replacement



| | Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|---|
| | Pointy | Round | Present | 1 |
| | Floppy | Not round | Absent | 0 |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

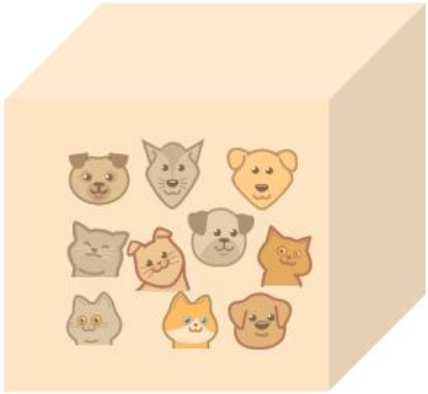
UNIVERSITY OF LEICESTER

# Sampling with Replacement



| | Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|---|
| | Pointy | Round | Present | 1 |
| | Floppy | Not round | Absent | 0 |
| | Pointy | Round | Absent | 1 |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

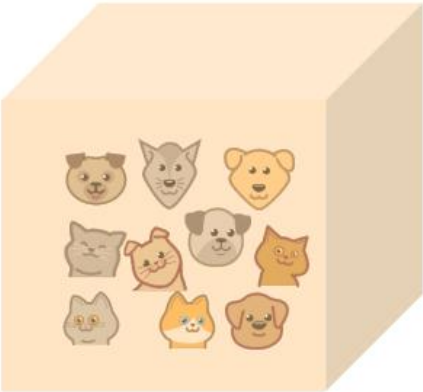
# Sampling with Replacement



| | Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|---|
| | Pointy | Round | Present | 1 |
| | Floppy | Not round | Absent | 0 |
| | Pointy | Round | Absent | 1 |
| | Pointy | Not round | Present | 0 |
| | Floppy | Not round | Absent | 0 |
| | Pointy | Round | Absent | 1 |
| | Pointy | Round | Present | 1 |
| | Floppy | Not round | Present | 1 |
| | Floppy | Round | Absent | 0 |
| | Pointy | Round | Absent | 1 |

UNIVERSITY OF LEICESTER

# Sampling with Replacement



| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | 1 |
| Floppy | Not round | Absent | 0 |
| Pointy | Round | Absent | 1 |
| Pointy | Not round | Present | 0 |
| Floppy | Not round | Absent | 0 |
| Pointy | Round | Absent | 1 |
| Pointy | Round | Present | 1 |
| Floppy | Not round | Present | 1 |
| Floppy | Round | Absent | 0 |
| Pointy | Round | Absent | 1 |

UNIVERSITY OF LEICESTER

# Agenda

- Decision trees
  - Tree ensembles
  - Sampling with replacement
  - **Random forest algorithm**

# Generating Tree Sample

Given training set of size $m$

# Generating Tree Sample

Given training set of size $m$

For $b$ = 1 to $B$:

# Generating Tree Sample

Given training set of size $m$

For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |

# Generating Tree Sample

Given training set of size $m$

For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |

# Generating Tree Sample

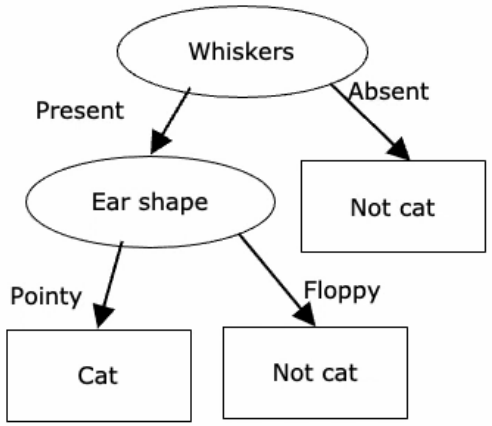
Given training set of size $m$

For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |



# Generating Tree Sample

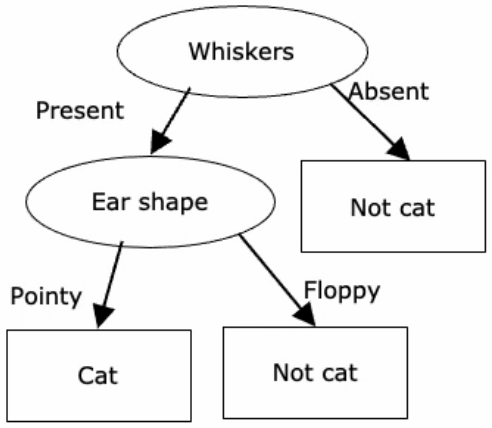
Given training set of size $m$

For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Pointy | Round | Absent | Yes |
| Floppy | Not Round | Absent | No |
| Floppy | Not Round | Absent | No |
| Pointy | Round | Absent | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Round | Present | Yes |

# Generating Tree Sample

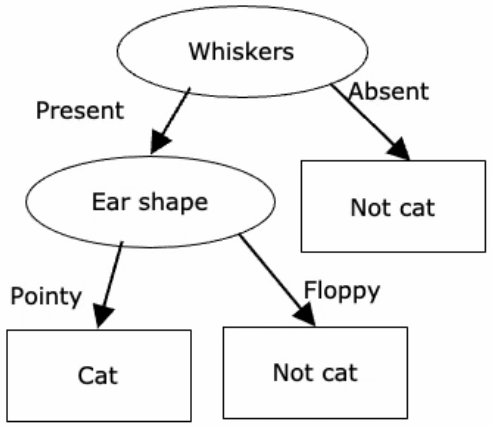
Given training set of size $m$

For $b$ = 1 to $B$:

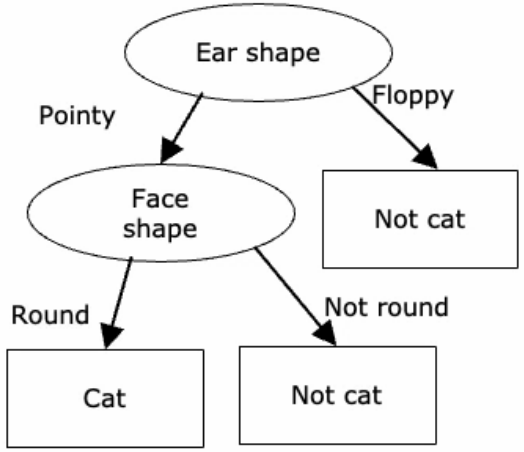
Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Pointy | Round | Absent | Yes |
| Floppy | Not Round | Absent | No |
| Floppy | Not Round | Absent | No |
| Pointy | Round | Absent | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Round | Present | Yes |




UNIVERSITY OF LEICESTER

# Generating Tree Sample

Given training set of size $m$

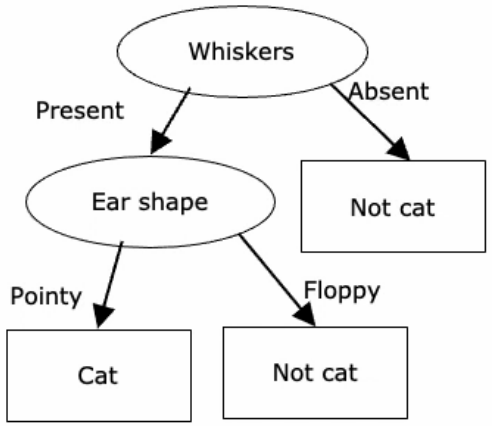
For $b$ = 1 to $B$:

**Number of samples = number of trees trained**

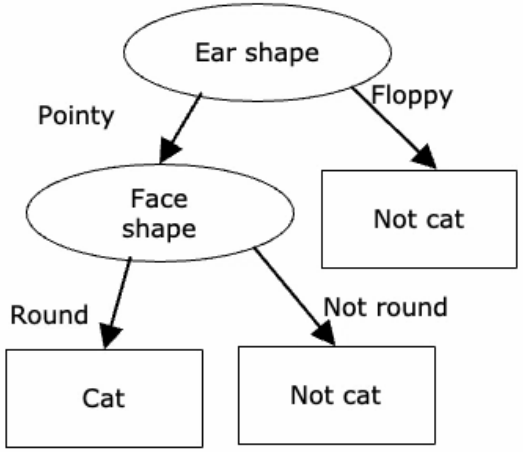
Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Pointy | Round | Absent | Yes |
| Floppy | Not Round | Absent | No |
| Floppy | Not Round | Absent | No |
| Pointy | Round | Absent | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Round | Present | Yes |



# Generating Tree Sample

Given training set of size $m$

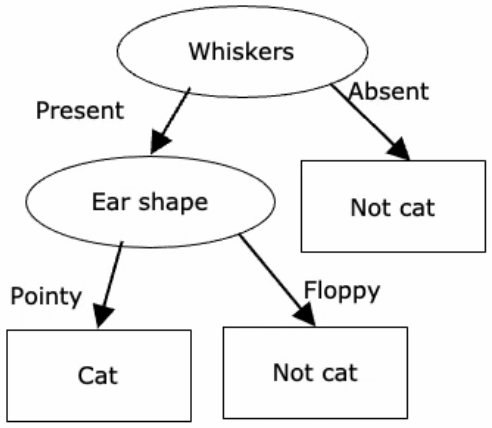
For $b$ = 1 to $B$:

**Number of samples = number of trees trained**

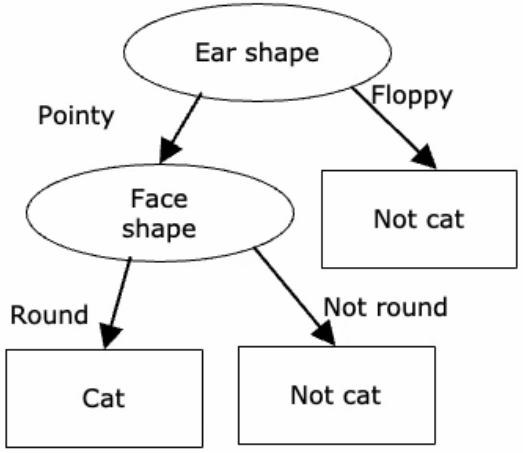
Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Pointy | Round | Absent | Yes |
| Floppy | Not Round | Absent | No |
| Floppy | Not Round | Absent | No |
| Pointy | Round | Absent | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Round | Present | Yes |




Bagged decision tree

UNIVERSITY OF LEICESTER

# Generating Tree Sample

Plausible number
**64 ~ 128**

Given training set of size $m$

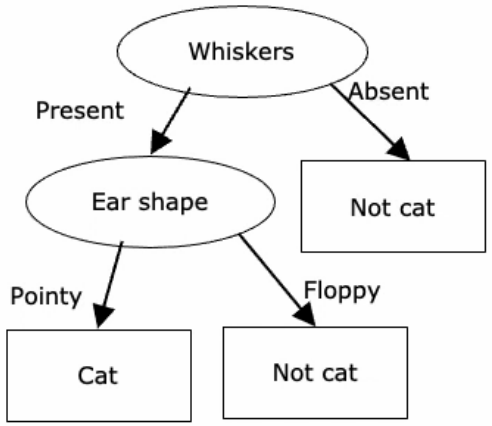
For $b$ = 1 to $B$:

**Number of samples = number of trees trained**

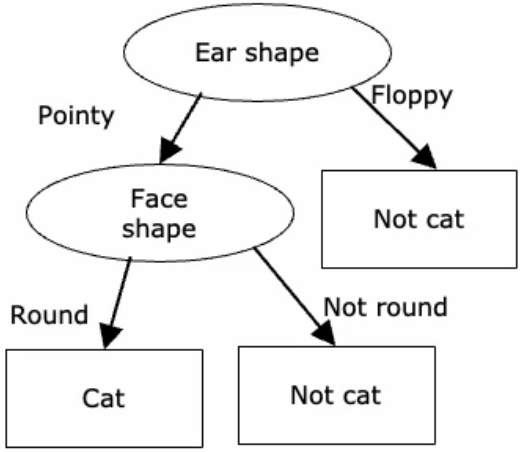
Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Pointy | Round | Absent | Yes |
| Floppy | Not Round | Absent | No |
| Floppy | Not Round | Absent | No |
| Pointy | Round | Absent | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Round | Present | Yes |




Bagged decision tree

UNIVERSITY OF LEICESTER

# Random Forest Algorithm

At each node, when choosing a feature to use to split,

if $n$ features are available,

# Random Forest Algorithm

At each node, when choosing a feature to use to split,

if $n$ features are available,

pick a random subset of $k < n$ features, and

# Random Forest Algorithm

At each node, when choosing a feature to use to split,

if $n$ features are available,

pick a random subset of $k < n$ features, and

allow the algorithm to only choose from that subset having $k$ features

UNIVERSITY OF LEICESTER

# Random Forest Algorithm

Added robustness!

At each node, when choosing a feature to use to split,

if $n$ features are available,

pick a random subset of $k < n$ features, and

allow the algorithm to only choose from that subset having $k$ features

# Random Forest Algorithm

Added robustness!

At each node, when choosing a feature to use to split,

if $n$ features are available,

pick a random subset of $k < n$ features, and

allow the algorithm to only choose from that subset having $k$ features

$$k = \sqrt{n}$$

# Agenda

- Decision trees
  - Tree ensembles
  - Sampling with replacement
  - Random forest algorithm
  - **Boosted trees**

# Boosted Trees

Given training set of size $m$

For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

Train a decision tree on the new dataset

# Boosted Trees

Given training set of size $m$

For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

But instead of picking from all examples with equal (*1/m*) probability, make it more likely to pick examples that the previously trained trees misclassify

Train a decision tree on the new dataset

# Boosted Trees

Given training set of size $m$

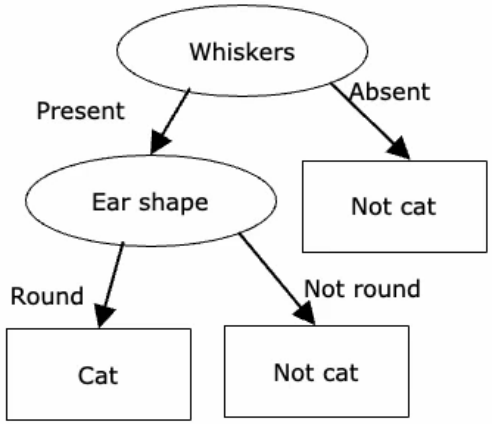
For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

But instead of picking from all examples with equal (*1/m*) probability, make it more likely to pick examples that the previously trained trees misclassify

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |



# Boosted Trees

Given training set of size $m$

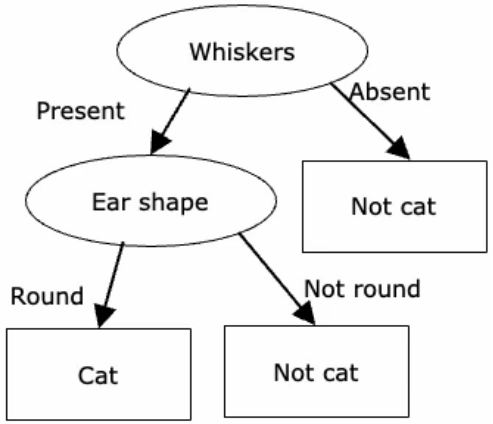
For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

But instead of picking from all examples with equal (*1/m*) probability, make it more likely to pick examples that the previously trained trees misclassify

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Prediction |
|---|---|---|---|
| Pointy | Round | Present | Cat ✅ |
| Floppy | Not Round | Present | Not cat ❌ |
| Floppy | Round | Absent | Not cat ✅ |
| Pointy | Not Round | Present | Not cat ✅ |
| Pointy | Round | Present | Cat ✅ |
| Pointy | Round | Absent | Not cat ❌ |
| Floppy | Not Round | Absent | Not cat ✅ |
| Pointy | Round | Absent | Not cat ❌ |
| Floppy | Round | Absent | Not cat ✅ |
| Floppy | Round | Absent | Not cat ✅ |

UNIVERSITY OF LEICESTER

# Boosted Trees

Given training set of size $m$

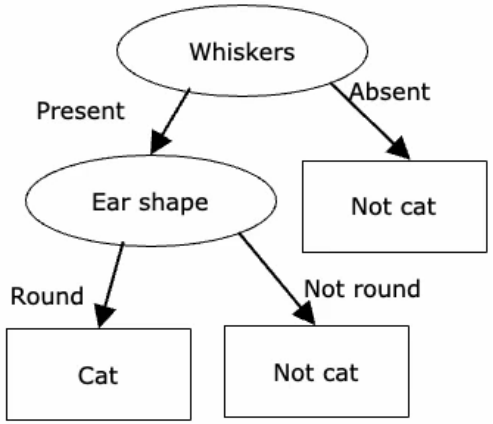
For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

But instead of picking from all examples with equal (*1/m*) probability, make it more likely to pick examples that the previously trained trees misclassify

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Prediction |
|---|---|---|---|
| Pointy | Round | Present | Cat ✅ |
| Floppy | Not Round | Present | Not cat ❌ |
| Floppy | Round | Absent | Not cat ✅ |
| Pointy | Not Round | Present | Not cat ✅ |
| Pointy | Round | Present | Cat ✅ |
| Pointy | Round | Absent | Not cat ❌ |
| Floppy | Not Round | Absent | Not cat ✅ |
| Pointy | Round | Absent | Not cat ❌ |
| Floppy | Round | Absent | Not cat ✅ |
| Floppy | Round | Absent | Not cat ✅ |

UNIVERSITY OF LEICESTER

# Boosted Trees

While training ***b*th** tree, higher probability of selecting training examples misclassified on previous ***b-1*** instances

Given training set of size $m$

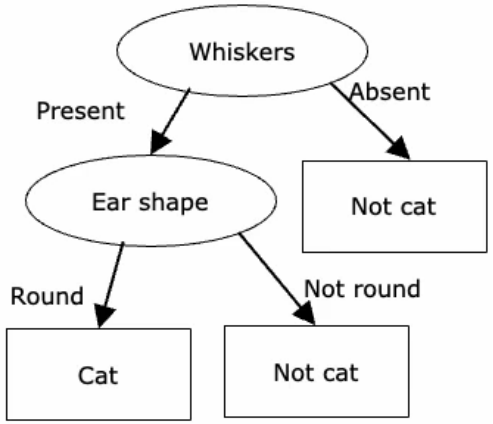
For $b$ = 1 to $B$:

Use sampling with replacement to create a new training set of size $m$

But instead of picking from all examples with equal (*1/m*) probability, make it more likely to pick examples that the previously trained trees misclassify

Train a decision tree on the new dataset

| Ear shape | Face shape | Whiskers | Cat |
|---|---|---|---|
| Pointy | Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Absent | No |
| Pointy | Round | Present | Yes |
| Pointy | Not Round | Present | Yes |
| Floppy | Round | Absent | No |
| Floppy | Round | Present | Yes |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Absent | No |
| Pointy | Not Round | Present | Yes |




| Ear shape | Face shape | Whiskers | Prediction |
|---|---|---|---|
| Pointy | Round | Present | Cat ✅ |
| Floppy | Not Round | Present | Not cat ❌ |
| Floppy | Round | Absent | Not cat ✅ |
| Pointy | Not Round | Present | Not cat ✅ |
| Pointy | Round | Present | Cat ✅ |
| Pointy | Round | Absent | Not cat ❌ |
| Floppy | Not Round | Absent | Not cat ✅ |
| Pointy | Round | Absent | Not cat ❌ |
| Floppy | Round | Absent | Not cat ✅ |
| Floppy | Round | Absent | Not cat ✅ |

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in **regularization** to prevent overfitting

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions (eg: Kaggle competitions)

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions (eg: Kaggle competitions)
- **LightGBM:** Faster XGBoost

# XGBoost (eXtreme Gradient Boosting)

- Open source implementation of boosted trees
- Fast efficient implementation
- Good choice of default splitting criteria and criteria for when to stop splitting
- Built in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions (eg: Kaggle competitions)
- **LightGBM:** Faster XGBoost
- **CatBoost:** Suitable for categorical features without one-hot encoding

# Agenda

- Decision trees
  - Tree ensembles
  - Sampling with replacement
  - Random forest algorithm
  - Boosted trees
  - **When to use?**

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast
  - Small decision trees may be human interpretable

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast
  - Small decision trees may be human interpretable
- **Neural Networks**
  - Works well on all types of data, including tabular (structured) and unstructured data

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast
  - Small decision trees may be human interpretable
- **Neural Networks**
  - Works well on all types of data, including tabular (structured) and unstructured data
  - May be slower than a decision tree

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast
  - Small decision trees may be human interpretable
- **Neural Networks**
  - Works well on all types of data, including tabular (structured) and unstructured data
  - May be slower than a decision tree
  - Works with transfer learning

# Decision Trees vs Neural Networks

- **Decision Trees and Tree ensembles**
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast
  - Small decision trees may be human interpretable
- **Neural Networks**
  - Works well on all types of data, including tabular (structured) and unstructured data
  - May be slower than a decision tree
  - Works with transfer learning
  - When building a system of multiple models working together, it might be easier to string together multiple neural networks

# Agenda

- Decision trees
  - Evaluating performance

# Evaluating Models

- Up to this stage, the focus has been on how models are built:
  - how decision trees split data,

# Evaluating Models

- Up to this stage, the focus has been on how models are built:
  - how decision trees split data,
  - why deep trees can fit training data extremely well,

# Evaluating Models

- Up to this stage, the focus has been on how models are built:
  - how decision trees split data,
  - why deep trees can fit training data extremely well,
  - how ensembles combine multiple trees to improve robustness.

# Evaluating Models

- Up to this stage, the focus has been on how models are built:
  - how decision trees split data,
  - why deep trees can fit training data extremely well,
  - how ensembles combine multiple trees to improve robustness.
- What has not yet been answered is a critical question: how do we know whether a trained model is actually reliable?

# What Does a "Good" Model Mean?

- A model is considered good **not** because it performs well on the **data it was trained** on, but because it performs well on **new, unseen data**.

# What Does a "Good" Model Mean?

- A model is considered good **not** because it performs well on the **data it was trained** on, but because it performs well on **new, unseen data**.
- The goal of machine learning is **not to memorise past examples**, but to **learn patterns that generalise beyond them**.

# What Does a "Good" Model Mean?

- A model is considered good **not** because it performs well on the **data it was trained** on, but because it performs well on **new, unseen data**.
- The goal of machine learning is **not to memorise past examples**, but to **learn patterns that generalise beyond them**.
- A good model therefore balances **two competing goals**:

# What Does a "Good" Model Mean?

- A model is considered good **not** because it performs well on the **data it was trained** on, but because it performs well on **new, unseen data**.
- The goal of machine learning is **not to memorise past examples**, but to **learn patterns that generalise beyond them**.
- A good model therefore balances **two competing goals**:
  - It should capture real structure in the data.

# What Does a "Good" Model Mean?

- A model is considered good **not** because it performs well on the **data it was trained** on, but because it performs well on **new, unseen data**.
- The goal of machine learning is **not to memorise past examples**, but to **learn patterns that generalise beyond them**.
- A good model therefore balances **two competing goals**:
  - It should capture real structure in the data.
  - It should avoid fitting details that are unlikely to repeat in new data.

# What Does a "Good" Model Mean?

- A model is considered good **not** because it performs well on the **data it was trained** on, but because it performs well on **new, unseen data**.
- The goal of machine learning is **not to memorise past examples**, but to **learn patterns that generalise beyond them**.
- A good model therefore balances **two competing goals**:
  - It should capture real structure in the data.
  - It should avoid fitting details that are unlikely to repeat in new data.
- This balance is at the heart of model **evaluation** and **validation**.

# Model Evaluation Pipeline

- When a model is trained, it is exposed to examples along with their correct outcomes.
  - The model's goal is to adjust itself to fit the patterns present in that data.

# Model Evaluation Pipeline

- When a model is trained, it is exposed to examples along with their correct outcomes.
  - The model's goal is to adjust itself to fit the patterns present in that data.
- If we then judge the model using the same data, we are effectively asking it questions it has already seen.
  - To assess whether a model has learned something meaningful, we must test it on unseen data.

# Model Evaluation Pipeline

- When a model is trained, it is exposed to examples along with their correct outcomes.
  - The model's goal is to adjust itself to fit the patterns present in that data.
- If we then judge the model using the same data, we are effectively asking it questions it has already seen.
  - To assess whether a model has learned something meaningful, we must test it on unseen data.
- Model evaluation is not a single calculation but a process. At a high level, this process involves
  - preparing the data,
  - training the model on one part of the data,
  - and then measuring how well it performs on a different part that it has not seen before.

# Model Evaluation Pipeline

**The goal is to see how well training results translate into real-world performance.**

- When a model is trained, it is exposed to examples along with their correct outcomes.
  - The model's goal is to adjust itself to fit the patterns present in that data.
- If we then judge the model using the same data, we are effectively asking it questions it has already seen.
  - To assess whether a model has learned something meaningful, we must test it on unseen data.
- Model evaluation is not a single calculation but a process. At a high level, this process involves
  - preparing the data,
  - training the model on one part of the data,
  - and then measuring how well it performs on a different part that it has not seen before.

# Why Evaluation Alone Is Not Enough

- When working with tree-based models, there are many possible versions of a model, each created by making different design choices including:
  - how deep the tree is allowed to grow, or
  - how many samples are required before making a split.

# Why Evaluation Alone Is Not Enough

- When working with tree-based models, there are many possible versions of a model, each created by making different design choices including:
  - how deep the tree is allowed to grow, or
  - how many samples are required before making a split.
- For ensembles, choices include:
  - how many trees are used, and
  - how complex each tree is.

# Why Evaluation Alone Is Not Enough

- When working with tree-based models, there are many possible versions of a model, each created by making different design choices including:
  - how deep the tree is allowed to grow, or
  - how many samples are required before making a split.
- For ensembles, choices include:
  - how many trees are used, and
  - how complex each tree is.
- If we evaluate only one version of a model on unseen data, we miss an important question:
  - was this the right version to evaluate in the first place?

# Why Evaluation Alone Is Not Enough

- When working with tree-based models, there are many possible versions of a model, each created by making different design choices including:
  - how deep the tree is allowed to grow, or
  - how many samples are required before making a split.
- For ensembles, choices include:
  - how many trees are used, and
  - how complex each tree is.
- If we evaluate only one version of a model on unseen data, we miss an important question:
  - was this the right version to evaluate in the first place?
- Evaluation can tell us how a chosen model performs, but it cannot tell us whether a better choice existed among alternative configurations.

# Why Evaluation Alone Is Not Enough

- When working with tree-based models, there are many possible versions of a model, each created by making different design choices including:
  - how deep the tree is allowed to grow, or
  - how many samples are required before making a split.
- For ensembles, choices include:
  - how many trees are used, and
  - how complex each tree is.
- If we evaluate only one version of a model on unseen data, we miss an important question:
  - was this the right version to evaluate in the first place?
- Evaluation can tell us how a chosen model performs, but it cannot tell us whether a better choice existed among alternative configurations.

**This is where the need for validation arises.**

# Why Evaluation Alone Is Not Enough

- When working with tree-based models, there are many possible versions of a model, each created by making different design choices including:
  - how deep the tree is allowed to grow, or
  - how many samples are required before making a split.

**Hyperparameters**

- For ensembles, choices include:
  - how many trees are used, and
  - how complex each tree is.
- If we evaluate only one version of a model on unseen data, we miss an important question:
  - was this the right version to evaluate in the first place?
- Evaluation can tell us how a chosen model performs, but it cannot tell us whether a better choice existed among alternative configurations.

**This is where the need for validation arises.**

# Training vs Validation vs Test Data

- We use three subsets and each subset of the data has a specific role.
  - The training data is used to learn patterns and build the structure of the tree or ensemble.
  - The validation data is used to compare different versions of a model and decide which configuration works best.
  - The test data is reserved for a final, unbiased assessment of the chosen model.

# Training vs Validation vs Test Data

- We use three subsets and each subset of the data has a specific role.
  - The training data is used to learn patterns and build the structure of the tree or ensemble.
  - The validation data is used to compare different versions of a model and decide which configuration works best.
  - The test data is reserved for a final, unbiased assessment of the chosen model.
- For decision trees, complexity is often reflected in how deep the tree is allowed to grow.
  - Shallow trees make only a few splits and capture broad patterns,
  - while deeper trees create many splits and capture fine details.

# Training vs Validation vs Test Data

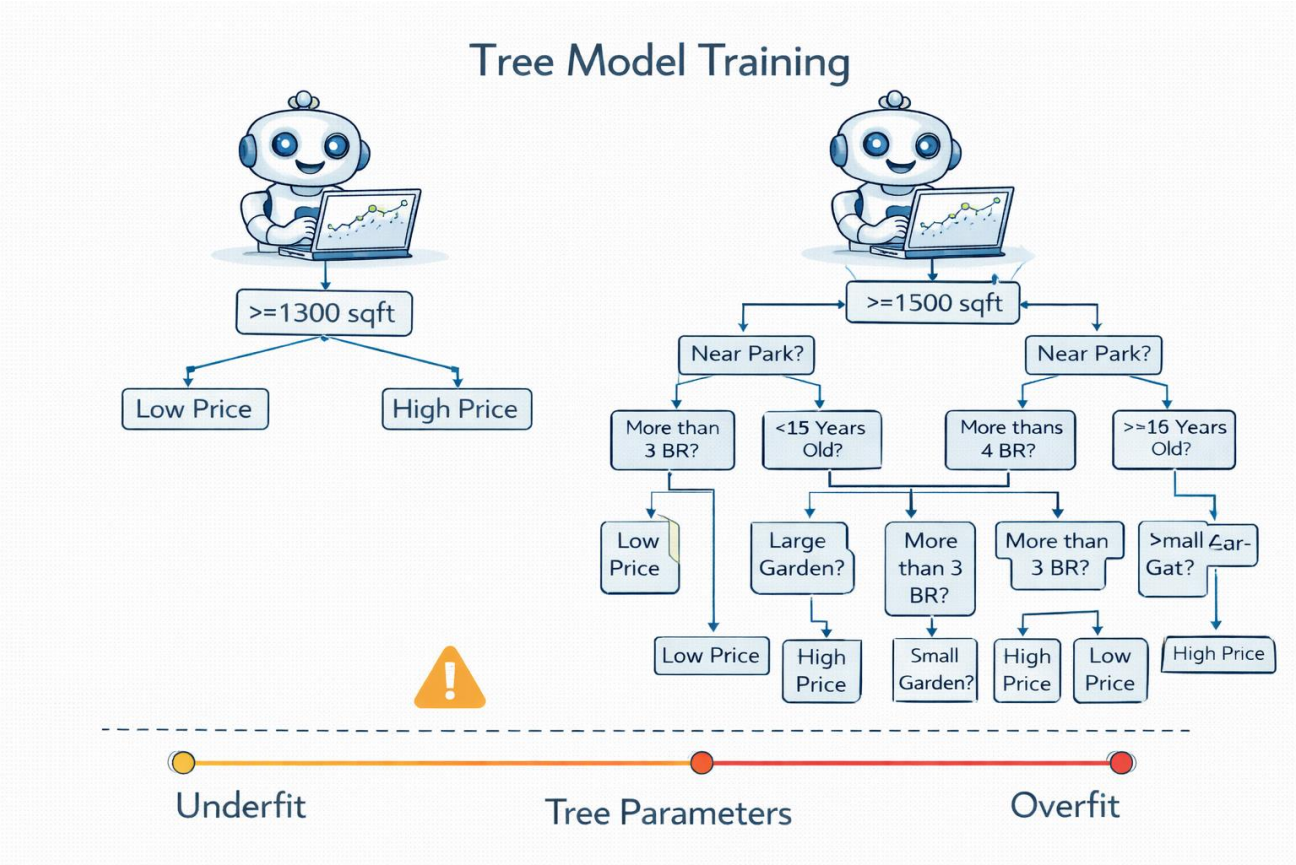
- We use three subsets and each subset of the data has a specific role.
  - The training data is used to learn patterns and build the structure of the tree or ensemble.
  - The validation data is used to compare different versions of a model and decide which configuration works best.
  - The test data is reserved for a final, unbiased assessment of the chosen model.
- For decision trees, complexity is often reflected in how deep the tree is allowed to grow.
  - Shallow trees make only a few splits and capture broad patterns,
  - while deeper trees create many splits and capture fine details.
- Validation provides a systematic way to compare these alternatives.
  - By training multiple versions of a tree with different levels of complexity and evaluating each one on validation data, we can observe how complexity affects performance on data the model has not seen during training.

# Training vs Validation vs Test Data

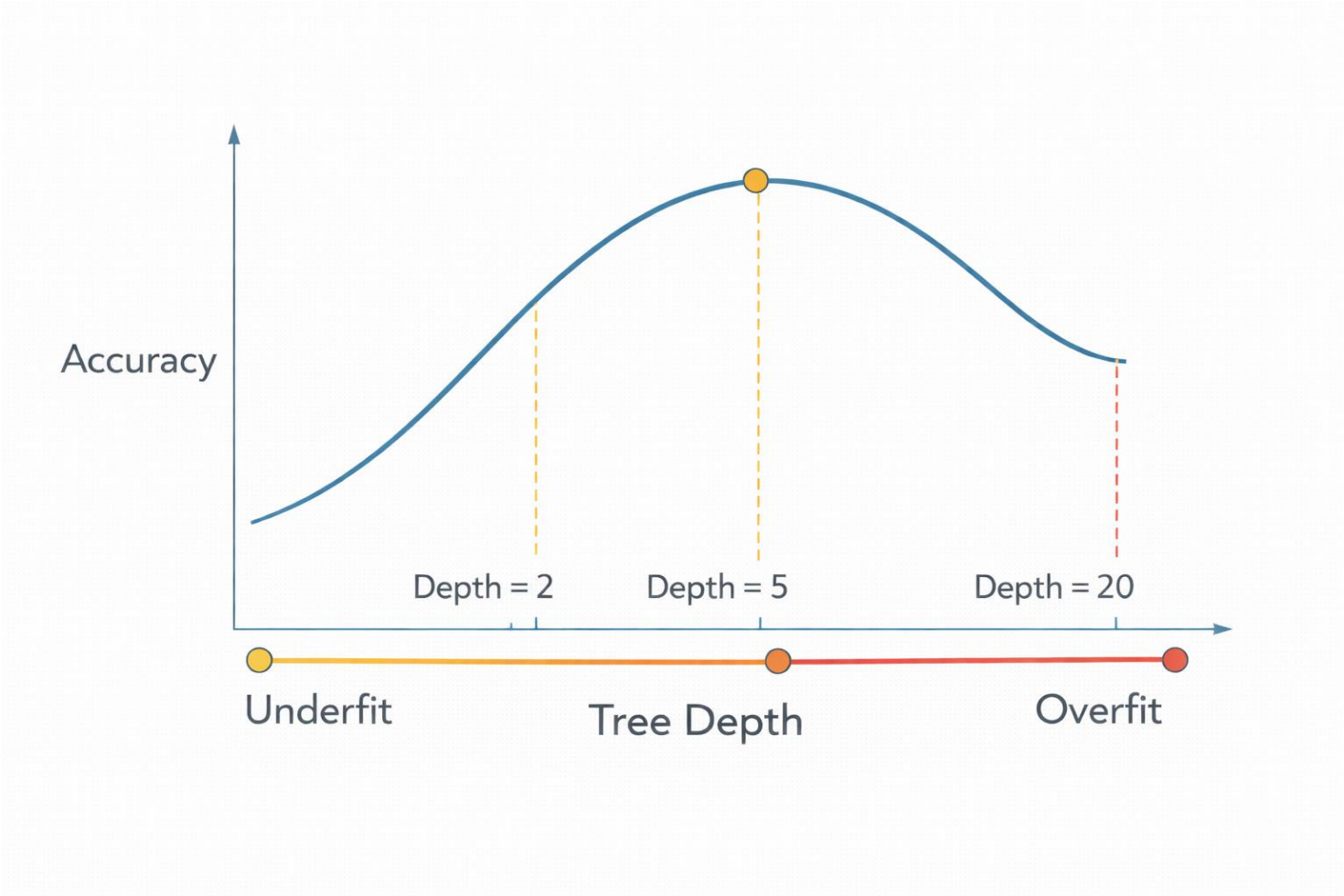
- We use three subsets and each subset of the data has a specific role.
  - The training data is used to learn patterns and build the structure of the tree or ensemble.
  - The validation data is used to compare different versions of a model and decide which configuration works best.
  - The test data is reserved for a final, unbiased assessment of the chosen model.
- For decision trees, complexity is often reflected in how deep the tree is allowed to grow.
  - Shallow trees make only a few splits and capture broad patterns,
  - while deeper trees create many splits and capture fine details.
- Validation provides a systematic way to compare these alternatives.
  - By training multiple versions of a tree with different levels of complexity and evaluating each one on validation data, we can observe how complexity affects performance on data the model has not seen during training.

**This process helps identify a balance point where the model is complex enough to capture meaningful structure but not so complex that it starts fitting noise.**

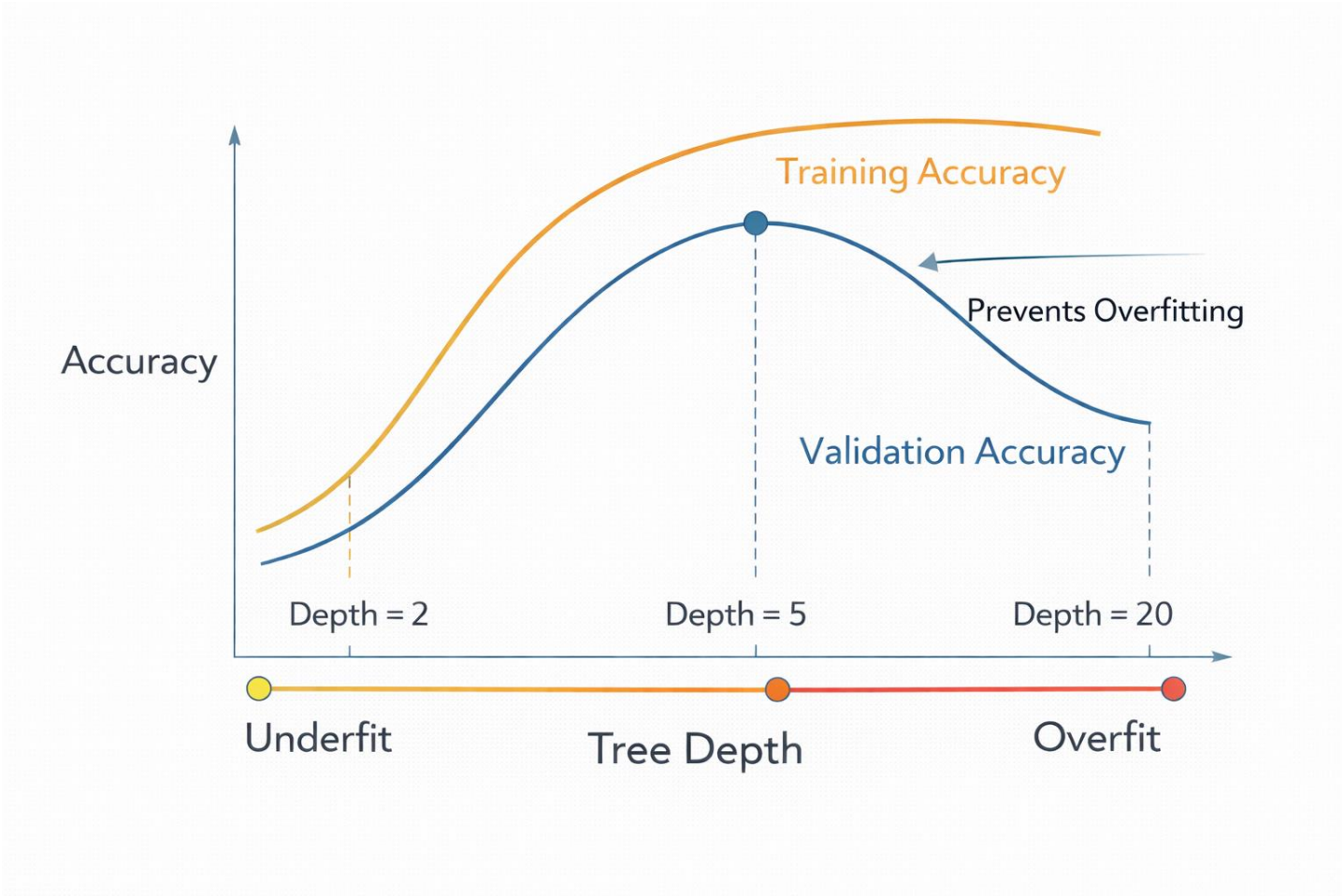
# Validation and Overfitting



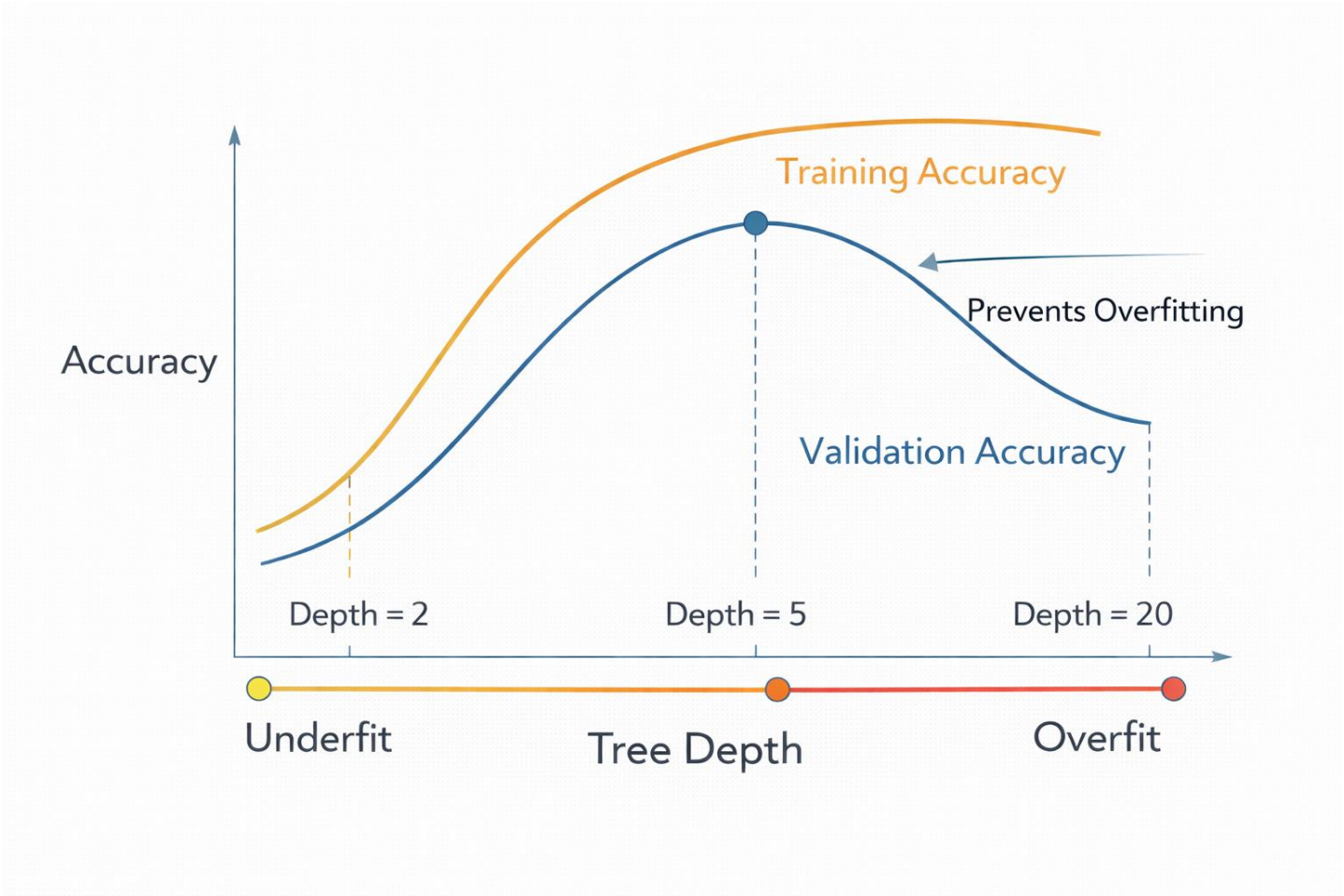
# Validation and Overfitting



# Validation and Overfitting



# Validation and Overfitting




**Bias, Variance Trade-off!**